

mitabl Admin Panel - Plan, Design & Architecture

By: Usman Saleem

28-Feb-2026

Decision Summary

- Keep the public marketing website public and unauthenticated. It remains focused on brand/commercial pages.
- Replace Salesforce capabilities with first-party modules in the Laravel backend + Filament admin portal.
- Build internal web apps using Filament PHP for:
 - Customer Service CRM
 - Operations (certification/approval)
 - Platform Administration (configuration, policies, health, controls)
- For mitabl every feature Salesforce currently does (certificate review, user lookup, support workflows) maps to Filament primitives:
 - Resource = CRUD manager for one model
 - Action = contextual operation with modal/form
 - Widget = dashboard card/chart/alert
- The admin panel will share the same DB and domain models already used by mobile APIs (User , Mikitchn , Certificate , Order , Payment , etc.).
- OUT OF SCOPE:
 - No web-app for the end-users for now. There will be only mobile app that will remain the end-user product channel.
 - Rebuilding mobile app frontend.
 - Replatforming payments away from Stripe.

Scope, Boundaries

- Salesforce replacement for:
 - Certificate approvals/workflows
 - Lead/pre-registration capture
 - Support case/ticket management
 - Internal user/kitchen lookup and operations workflows
- Internal portal for:

- Customer Service
 - Operations
 - Platform Admin/Super Admin
- Platform controls:
 - Configuration and policies
 - Health checks and status pages
 - Queue/job visibility and operational actions
 - Audit trail and governance
- Marketing Website
 - Marketing site remains public and catchy.

Current State - What Salesforce Does (today and will be replaced)

SF Capability	Current implementation	Status after this plan
Kitchen certificate approval	PUT /api/kitchen/{id}/certificate called from SF UI	Replaced by Filament CertificateResource with Approve/Reject actions
User lookup (mifoodi details)	GET /api/sales/mifoodi called from SF	Replaced by Filament UserResource
Pre-registration lead capture	POST /api/preregister -> SF Lead	Replaced by local pre_registrations + Filament
Contact/support case creation	POST /api/mobcontact -> SF Case	Replaced by local support_tickets + Filament
Kitchen profile sync (Account)	KitchenVerifiedToSales listener -> SF Account	Listener removed; data remains in mitabl DB
SF Account CRM ID storage	sales_kitchens table	Table dropped after cutover
SF OAuth token workflow	WebApiToCurlController::getAccToken()	Removed entirely

Architecture Overview

Repository Strategy

- Internal ops portal is part of the existing backend repo/codebase.
- Public marketing website remains in the existing marketing website repo.
- No new repo required for the ops portal in this phase.

Deployment Topology

Use separate deployable services/containers:

- marketing-web container
 - serves public pages only
 - no admin auth surface
- backend-api container
 - serves /api/* for mobile and intake APIs
 - can also host Filament routes unless split is desired
- ops-admin container (recommended split from backend runtime)
 - same Laravel codebase/artifact as backend, but admin-focused entry route and scaling profile
 - isolates admin runtime concerns and access controls
- shared dependencies:
 - DB
 - Redis/queue
 - object storage

Ingress/routing example (same domain):

- mitabl.com/* -> marketing-web
- mitabl.com/api/* -> backend-api
- mitabl.com/admin/* -> ops-admin (authenticated + allowlist/MFA/protected)
- Optional:
 - mitabl.com/ops/* for selected admin modules if we want path segmentation by function

Note:

- backend-api and ops-admin can be built from the same image with different runtime env/entrypoint.

Surface 1: Public Marketing Website (keep)

- Public/commercial pages only.
- No admin or CRM business logic.
- Contact/lead forms submit to backend intake APIs.

Surface 2: Backend Core (Laravel API, source of truth)

- Existing mobile APIs remain.
- New CRM and platform-admin modules live in same Laravel codebase.
- Events/jobs drive automation.

Surface 3: Internal Admin Portal (Filament at /admin)

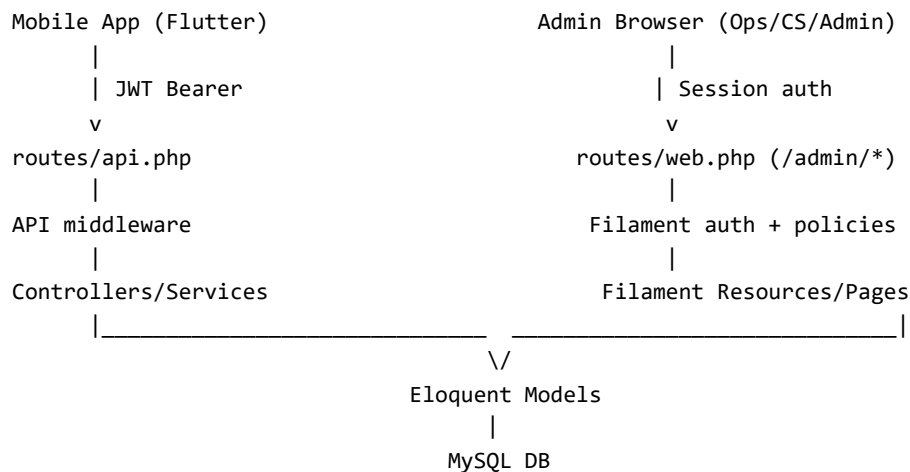
- Session-authenticated, role-gated.
- Multi-panel navigation by function:
 - Customer Service
 - Operations
 - Platform Admin

Where the Admin Panel Lives

The admin panel is not a separate application. It is mounted inside backend/ :

```
backend/
|-- app/
|   |-- Filament/                # admin panel code
|   |   |-- Resources/
|   |   |-- Pages/
|   |   `-- Widgets/
|   |-- Domain/                  # new domain services/workflows
|   |-- Http/Controllers/Api/    # existing mobile APIs + new intake APIs
|   `-- Models/                  # shared domain models
|-- database/
|   `-- migrations/
`-- routes/
    |-- api.php                  # mobile + intake APIs
    `-- web.php                  # marketing + admin auth routes
```

Request Flow - Parallel Entry Points



Admin panel does not call mobile API endpoints. It uses domain services + Eloquent directly.

Admin Authentication and Roles

Use separate admin identities from consumer/mobile users.

Admin Identity Model

- Recommended: `admin_users` table + `AdminUser` model.
- Separate auth guard/provider from API JWT user auth.
- Supports least-privilege and avoids session/JWT boundary mistakes.

RBAC Roles (Spatie)

Role	Core permissions
<code>super_admin</code>	Full access, IAM, policy/config changes, emergency ops
<code>platform_admin</code>	Config/policy/health, integrations, feature flags, observability controls
<code>operations</code>	Certification workflows, kitchen lifecycle, onboarding approvals
<code>customer_service</code>	Tickets, dispute handling, user/order lookup, limited override actions
<code>finance_readonly</code> (optional)	Read-only payments/refunds/reporting visibility

Permission Guardrails

- No implicit role inheritance; explicit permission mapping.
- High-risk actions require step-up confirmation:
 - Suspend account
 - Manual refund
 - Force status transitions
 - Policy changes affecting production behavior

Feature Modules

Module 1 - Dashboard (Home)

Purpose: real-time operational view at login.

Widgets: | Widget | Data source | Refresh | |---|---|---| | Users by role (Foodies/Cooks) |
users grouped by role_id | On load | | Active kitchens | mikitchns.status=1 | On load
| | Pending certificate approvals | certificates.status=0 | Poll 30s | | Open support
tickets | support_tickets.status in (open,in_progress,pending_user) | Poll 30s | | SLA
breach risk | tickets due in next N mins / overdue | Poll 30s | | Orders today |
orders.delivery_date=today | On load | | Revenue this month | payments joined with
confirmed orders | On load | | Queue health | failed jobs + queue latency snapshot | Poll
30s | | Integration health | mail/storage/FCM/Stripe status checks | Poll 30s | | Orders/day
chart (30d) | orders grouped by date | On load | | Registrations/day chart (30d) | users
grouped by date | On load |

Module 2 - Kitchen Certificate Management (replaces SF approval flow)

Resource: CertificateResource

Table:

- Kitchen, cook, phone, ABN, certificate number
- Upload date
- Document preview/download
- Status badge (Pending/Approved/Rejected)
- Reviewer, reviewed timestamp

Filters:

- Status
- Submission date range
- Kitchen/cook search

Actions:

- Approve
 - sets status=1
 - captures reviewed_by , reviewed_at
 - sends approved notification
- Reject
 - sets status=2
 - mandatory rejection_reason
 - captures reviewer metadata
 - sends rejection email + in-app notification

- Request Re-Submission (optional intermediate)
 - keeps ticket/workflow open for revised docs

Status values:

0 = Pending review
 1 = Approved
 2 = Rejected

Required fields:

```
$table->text('rejection_reason')->nullable();
$table->timestamp('reviewed_at')->nullable();
$table->foreignId('reviewed_by')->nullable()->constrained('admin_users');
```

Edge cases:

- Missing/invalid file type
- Multiple uploads during review
- Concurrent approvals by two agents
- Retry-safe notifications on queue retries

Replaces/removes:

- Salesforce approval endpoints/middleware
- Salesforce kitchen sync listener dependency

Module 3 - User Management

Resource: `UserResource`

Table:

- ID, name, email, phone
- Role(s) and verification
- Account status (active/suspended/deleted)
- Order count and last activity

Actions:

- View profile timeline (orders, tickets, flags, notes)
- Edit profile (restricted fields)
- Suspend/Unsuspend (with reason)
- Soft delete / restore path (if policy permits)

- Link to kitchen/orders/tickets

New fields:

```
$table->boolean('suspended')->default(false);  
$table->text('suspension_reason')->nullable();  
$table->timestamp('suspended_at')->nullable();
```

Edge cases:

- Prevent suspension of super admin accounts
- Prevent self-lockout for final super admin
- Audit every status change and reason

Module 4 - Kitchen Management

Resource: MikitchnResource

Table:

- Kitchen ID/name, cook, location
- Kitchen status
- Certificate status
- Dine-in/take-away flags
- Rating/orders summary

Actions:

- View kitchen profile, media gallery, menu, reviews
- Activate/Deactivate kitchen
- Edit kitchen metadata
- Embedded certificate review panel

Edge cases:

- Status transitions blocked when certificate invalid
- Existing open bookings while attempting deactivation
- Data quality checks for geolocation and address fields

Module 5 - Order Management

Resource: OrderResource

Table:

- Order ID, kitchen, customer
- Order/delivery dates
- Type (dine-in/take-away)
- Total, paid flag, refund state
- Status badge

Actions:

- View full order detail and payment metadata
- View cancel reason and timeline
- Manual override for exceptional cases (permission-gated)
- Trigger full refund via controlled service endpoint

Status reference (confirm exact mapping in code):

```
1 = Completed
2 = Requested
3 = Confirmed
4 = Cancelled
```

Edge cases:

- Prevent duplicate refund execution
- Idempotent admin actions
- Mandatory rationale for manual override

Module 6 - Support Ticket Management (replaces SF Cases)

New tables:

- support_tickets
- support_ticket_messages
- support_ticket_attachments (recommended)
- support_ticket_events (recommended immutable audit stream)

Core ticket schema (minimum):

```
Schema::create('support_tickets', function (Blueprint $table) {
    $table->id();
    $table->string('ticket_number')->unique(); // TKT-000001
    $table->foreignId('user_id')->nullable()->constrained('users');
```

```

$table->string('requester_name')->nullable();
$table->string('requester_email');
$table->string('requester_phone')->nullable();
$table->string('subject');
$table->text('description');
$table->string('source'); // mobile_app, website, admin
$table->string('category'); // order_dispute,payment,account,
$table->string('priority')->default('normal');
$table->string('status')->default('open'); // open,in_progress,pending_user,
$table->foreignId('assigned_to')->nullable()->constrained('admin_users');
$table->foreignId('order_id')->nullable()->constrained('orders');
$table->foreignId('mitchn_id')->nullable()->constrained('mitchns');
$table->timestamp('first_response_due_at')->nullable();
$table->timestamp('resolution_due_at')->nullable();
$table->timestamp('first_responded_at')->nullable();
$table->timestamp('resolved_at')->nullable();
$table->timestamps();
$table->index(['status', 'priority', 'assigned_to']);
});

```

Thread messages:

```

Schema::create('support_ticket_messages', function (Blueprint $table) {
    $table->id();
    $table->foreignId('ticket_id')->constrained('support_tickets')->cascadeOnDelete();
    $table->string('sender_type'); // user, admin, system
    $table->unsignedBigInteger('sender_id')->nullable();
    $table->text('message');
    $table->boolean('is_internal_note')->default(false);
    $table->timestamps();
});

```



Resource: SupportTicketResource

Capabilities:

- Queue/list with SLA-aware sorting
- Assign/reassign ownership
- Change status/priority/category
- Public reply vs internal note
- Merge duplicate tickets
- Link to user/order/kitchen context
- Resolve/close with required resolution summary

CRM edge cases to support:

- Anonymous requester handling
- Duplicate detection (same requester + similar subject in short time)

- Spam/abuse throttling and spam state
- Collision handling when two agents edit same ticket
- SLA breach alerts/escalation routing
- Attachment size/type malware scanning policy
- PII redaction policy for exports and notifications

Module 7 - Pre-Registration / Lead Management (replaces SF Leads)

New table: pre_registrations

```
Schema::create('pre_registrations', function (Blueprint $table) {
    $table->id();
    $table->string('first_name');
    $table->string('last_name');
    $table->string('email')->nullable();
    $table->string('phone')->nullable();
    $table->string('city')->nullable();
    $table->string('interested_as'); // cook, foodie, both
    $table->string('source'); // website, referral, campaign, admin
    $table->string('status')->default('new'); // new, contacted, converted, disqualif
    $table->text('notes')->nullable();
    $table->foreignId('followed_up_by')->nullable()->constrained('admin_users');
    $table->timestamp('followed_up_at')->nullable();
    $table->timestamps();
    $table->index(['status', 'source']);
});
```

Resource: PreRegistrationResource

Actions:

- Mark contacted/converted/disqualified/spam
- Add follow-up notes
- Link conversion to created user account when available

Edge cases:

- Duplicate lead ingestion
- Email/phone normalization
- Consent and communication preference capture

Module 8 - Promo Code Management

Resource: `PromoCodeResource` over existing `promo_codes` .

Actions:

- Create/edit/deactivate
- Usage history
- Guardrails for invalid overlapping rules

Module 9 - Financial Overview (Read-Only + controlled refund action)

Resource: `PaymentResource`

Capabilities:

- Visibility into payment/refund states
- Link to order/user/kitchen
- External Stripe dashboard link (no secrets exposed)
- Controlled full refund action via service endpoint and strict permissions

Constraints:

- No direct key handling in UI
- All mutations idempotent and audited

Module 10 - Platform Admin: Configuration & Policies

Purpose: replace Salesforce admin/process controls and centralize operational governance.

Resources/pages:

- `PlatformSettingsPage`
 - business toggles (feature flags)
 - support defaults (priority, routing)
 - approval requirements
 - maintenance banner / operational notices
- `PolicyResource`
 - SLA policy (first response, resolution windows)
 - escalation policy
 - suspension policy
 - refund policy constraints

- document retention policy
- TemplateResource
 - email templates for ticket/certification flows
 - notification templates

Recommended data model:

- platform_settings (key/value, typed, versioned)
- policies (name, version, json, effective_at, active)
- policy_change_log (immutable audit)

Edge cases:

- Policy draft/publish workflow
- Rollback to previous policy version
- Validation by JSON schema before activation
- Blast-radius warning for high-impact policy changes

Module 11 - Platform Admin: Health, Operations, and Observability

Purpose: operations control plane for platform stability.

Resources/pages:

- SystemHealthPage
 - DB connectivity/latency
 - Redis/queue health
 - mail provider status
 - storage disk status
 - Stripe and FCM connectivity checks
 - scheduler heartbeat and failed cron detection
- QueueOpsPage
 - failed jobs list
 - retry/discard/requeue actions
 - queue depth and age
- IntegrationLogsPage
 - outbound webhook/API calls
 - response status and error body snapshots
 - retry attempts
- AuditLogResource
 - who changed what, before/after snapshots, request metadata

Operational edge cases:

- queue saturation
- poison-message retries
- dead-letter handling
- partial outage mode (degraded features)
- incident annotation and postmortem link fields

Module 12 - Internal Notes, Tags, Watchers, and Collaboration

Cross-module productivity features:

- Internal notes on users, kitchens, orders, tickets
- Tagging taxonomy (`fraud_risk` , `vip` , `repeat_issue` , etc.)
- Watchers/subscriptions for ticket updates
- Mentions in internal comments (future enhancement)

Database Migrations Required

Migration	Purpose
<code>add_rejected_fields_to_certificates</code>	<code>rejection_reason</code> , <code>reviewed_at</code> , <code>reviewed_by</code> , <code>status</code> support for rejected
<code>add_suspended_to_users</code>	<code>suspended</code> , <code>suspension_reason</code> , <code>suspended_at</code>
<code>create_support_tickets</code>	Ticket master table
<code>create_support_ticket_messages</code>	Ticket conversation threads
<code>create_support_ticket_attachments</code>	Attachment metadata
<code>create_support_ticket_events</code>	Immutable event trail
<code>create_pre_registrations</code>	Lead replacement table
<code>create_platform_settings</code>	Runtime platform settings
<code>create_policies</code>	Versioned policy definitions
<code>create_policy_change_log</code>	Policy audit trail
<code>create_admin_users</code>	Separate admin identity table

Migration	Purpose
drop_sales_kitchens	Remove SF mapping table after cutover

New / Modified Code Components

New Filament Resources

```
app/Filament/Resources/
|-- CertificateResource.php
|-- UserResource.php
|-- MikitchnResource.php
|-- OrderResource.php
|-- SupportTicketResource.php
|-- PreRegistrationResource.php
|-- PaymentResource.php
|-- PromoCodeResource.php
|-- PolicyResource.php
|-- AuditLogResource.php
`-- TemplateResource.php
```

New Filament Widgets

```
app/Filament/Widgets/
|-- PlatformStatsOverview.php
|-- OrdersPerDayChart.php
|-- NewUsersChart.php
|-- PendingCertificatesWidget.php
|-- OpenTicketsWidget.php
|-- SlaBreachWidget.php
|-- QueueHealthWidget.php
`-- IntegrationHealthWidget.php
```

New Filament Pages

```
app/Filament/Pages/
|-- Dashboard.php
|-- PlatformSettingsPage.php
|-- SystemHealthPage.php
`-- QueueOpsPage.php
```

New Models

```
app/Models/
|-- AdminUser.php
```

```
|-- SupportTicket.php
|-- SupportTicketMessage.php
|-- SupportTicketAttachment.php
|-- SupportTicketEvent.php
|-- PreRegistration.php
|-- PlatformSetting.php
|-- Policy.php
`-- PolicyChangeLog.php
```

New Mailables

```
app/Mail/
|-- CertificateRejected.php
|-- SupportTicketConfirmation.php
|-- SupportTicketReply.php
|-- PreRegistrationAcknowledgement.php
`-- SlaEscalationAlert.php
```

New/Modified API Endpoints

Method	Route	Change
POST	/api/support/ticket	New ticket intake (replaces SF case path)
GET	/api/support/ticket/{id}	User-side status lookup (scoped access)
POST	/api/preregister	Modified to store local lead
POST	/api/mobcontact	Kept as deprecated alias to /api/support/ticket during transition
PUT	/api/kitchen/{id}/certificate	Removed (SF inbound)
GET	/api/sales/mifoodi	Removed (SF inbound)

Backward compatibility:

- Keep /api/mobcontact alias for one release window.
- Emit deprecation headers/log warning.
- Remove after client updates are completed.

Code to Delete (Salesforce removal)

```
app/Listeners/KitchenVerifiedToSales.php
app/Http/Controllers/Api/Sales/...
app/Http/Middleware/SalesForce.php
```


app/Models/SalesKitchen.php
Salesforce logic inside WebApiToCurlController
EventServiceProvider SF listener bindings
Kernel salesforce middleware alias
SF route group in routes/api.php

Note: remove KitchenVerified event only if no non-SF listeners remain.

Env Variables to Remove

SALES_AUTH
SF_CLIENT_ID
SF_CLIENT_SECRET
SF_USERNAME
SF_PASSWORD

Salesforce Dependency Eradication Checklist (Mandatory)

Goal: remove Salesforce completely while preserving existing mitabl behavior for cooks, foodies, ops, and support.

1) Contract Parity Matrix

- Define old -> new ownership for each Salesforce capability and endpoint:
 - Certificate approval: PUT /api/kitchen/{id}/certificate (SF inbound) -> Filament CertificateResource actions.
 - User lookup: GET /api/sales/mifoodi (SF inbound) -> Filament UserResource search.
 - Lead intake: POST /api/preregister (SF outbound) -> local pre_registrations .
 - Support intake: POST /api/mobcontact (SF outbound) -> local support_tickets via compatibility alias.
 - Kitchen sync: KitchenVerifiedToSales listener -> removed, local DB remains source of truth.
- For each mapping, define acceptance tests and data invariants before deletion.

2) Compatibility and No-Impact Guardrails

- Preserve client-facing API contracts during transition:
 - keep request/response shape and status codes for /api/preregister and /api/mobcontact .
 - keep /api/mobcontact alias until all clients are on the new ticket flow.
- Add deprecation headers and structured logs for alias usage.
- Add idempotency protection for intake endpoints to avoid duplicate ticket/lead records.

- Add background retries only for internal notifications; do not make intake synchronous.

3) Data Integrity and Migration

- Backfill or map required Salesforce-originated operational fields into local tables before cutover.
- Create one-time reconciliation job:
 - detect duplicates in leads/tickets,
 - normalize email/phone,
 - enforce foreign key consistency for ticket links (`user_id` , `order_id` , `mikitchn_id`).
- Produce pre/post cutover record counts and mismatch reports for sign-off.

4) Security and Secret Cleanup

- Remove hardcoded Salesforce tokens/credentials from source code.
- Remove Salesforce secrets from env and secret manager after cutover completion.
- Rotate any credentials that were previously committed.
- Add CI secret scanning gate to block token reintroduction.

5) Code and Infrastructure Removal

- Delete:
 - `KitchenVerifiedToSales` listener and event binding,
 - Salesforce middleware and middleware alias,
 - Salesforce route group and controllers,
 - `sales_kitchens` model/table after validation window,
 - Salesforce helper logic from `WebApiToCurlController` .
- Remove Salesforce-specific runbooks, dashboards, and alerts; replace with local CRM observability.

6) Cutover Exit Criteria (Must Pass)

- 0 unresolved P1/P2 defects in certificate, support, and lead flows.
- 100% pass rate on contract parity tests for legacy touched endpoints.
- No increase in failed intake requests or ticket creation latency vs baseline.
- No remaining Salesforce calls in runtime logs for 7 consecutive days.

IMPLEMENTATION PLAN

Implementation Tasks

Phase 0 - Repository and Deployment Foundations (Week 0)

#	Task	Notes
0.1	Finalize repo boundaries	backend owns ops portal; marketing remains separate repo
0.2	Define same-domain ingress routes	/ marketing, /api/* backend, /admin/* ops portal
0.3	Decide runtime split	one shared image with 2 services (backend-api , ops-admin)
0.4	Configure TLS and path-based gateway rules	certs + HSTS + strict rules on /admin/*
0.5	Configure admin network controls	IP allowlist/VPN and MFA policy
0.6	Add deployment environments	dev, staging, prod parity for all 3 services
0.7	Add secrets management plan	remove hardcoded secrets, use vault/secret store

Phase 0.5 - Platform Hardening Baseline (Week 0, Mandatory Gate)

Purpose: remove known production blockers before building Filament modules. No feature/module work should start until this phase is complete.

#	Task	Notes
0.5.1	Upgrade PHP runtime baseline to 8.2+	Update runtime images/hosts and composer platform settings; validate all environments use the same minor line
0.5.2	Upgrade Laravel from 8 to 11	Execute incremental framework upgrade path with dependency compatibility checks and smoke tests after each major step
0.5.3	Upgrade mobile toolchain to Dart 3+	Update Flutter/Dart SDK constraints, refresh locked dependencies, and resolve null-safety/type breakages

#	Task	Notes
0.5.4	Enforce asynchronous queue in non-test environments	Set <code>QUEUE_CONNECTION=redis</code> for dev/staging/prod; keep <code>sync</code> only for selected local/test cases
0.5.5	Deploy dedicated queue workers	Run managed workers (<code>queue:work</code> /Horizon) with restart/health checks, retry policy, and dead-letter handling
0.5.6	Switch cache backend to Redis	Set <code>CACHE_DRIVER=redis</code> (or <code>CACHE_STORE=redis</code> on newer Laravel config style) and validate Redis connectivity/failover
0.5.7	Add cache wrappers for discovery endpoints	Add <code>Cache::remember</code> to nearest/top-rated/recommended discovery queries with scoped cache keys and short TTL
0.5.8	Remove wasteful double-fetch pagination patterns	Replace <code>get()->count()</code> on full result sets with query-level counts/pagination metadata
0.5.9	Add cache invalidation hooks	Invalidate discovery caches on kitchen/profile/certificate/review/menu changes via events/listeners
0.5.10	Extract payment logic from <code>StripeTrait</code> into <code>PaymentService</code>	Use container-injected services in controllers/listeners; remove direct SDK initialization from controllers
0.5.11	Introduce core service layer for large controllers	Create <code>AuthService</code> , <code>KitchenService</code> , <code>OrderService</code> , <code>DiscoveryService</code> ; thin controllers to validate + delegate
0.5.12	Remove Salesforce secrets and static tokens from source	Delete hardcoded OAuth/token values; move remaining integration secrets to secure env/secret manager during transition
0.5.13	Add observability for async + cache behavior	Track queue latency, failed jobs, cache hit ratio, and p95 endpoint latency for discovery APIs
0.5.14	Add regression tests for hardening work	Feature tests for async behavior, discovery response parity, cache invalidation, and key payment flows
0.5.15	Define hardening exit criteria	Gate: queue async confirmed, Redis cache live, no hardcoded secrets, discovery p95 improved, no P1 regressions

Phase 1 - Foundation (Week 1)

#	Task	Notes
1.1	Install Filament panel	<code>composer require filament/filament:^3.0</code>
1.2	Scaffold admin panel	<code>php artisan filament:install --panels</code>
1.3	Create <code>AdminUser</code> model/migration/seeder	Separate from mobile <code>User</code>
1.4	Install Spatie permission	Define role/permission matrix
1.5	Configure panel branding	Logo/colors/navigation
1.6	Add certificate migration changes	reject metadata fields
1.7	Add user suspension fields	
1.8	Create support ticket tables	tickets/messages/events/attachments
1.9	Create pre-registration table	
1.10	Create platform settings/policy tables	

Phase 2 - Core Resources (Weeks 1)

#	Task	Notes
2.1	Build <code>CertificateResource</code> with approve/reject	include concurrency protection
2.2	Build certificate mail/notification templates	queue-based
2.3	Build <code>UserResource</code> with suspension controls	audited actions
2.4	Build <code>MikitchnResource</code> with embedded cert panel	
2.5	Build <code>OrderResource</code> and controlled refund action	idempotent
2.6	Build <code>PromoCodeResource</code>	

Phase 3 - CRM (Weeks 1-2)

#	Task	Notes
3.1	Build ticket models + relationships + indexes	include SLA fields, assignment, linkage to user/order/kitchen
3.2	Build SupportTicketResource with full agent workflow	inbox, assignment, priority, status machine, merge/split, resolution summary
3.3	Build intake and conversation APIs	POST /api/support/ticket , GET /api/support/ticket/{id} , reply endpoints, scoped auth
3.4	Build spam/abuse protections for CRM intake	rate limits, honeypot/captcha hooks, duplicate detection
3.5	Build ticket automations	SLA timers, breach escalation jobs, queue-driven notifications
3.6	Build PreRegistrationResource with operator workflow	triage queue, status transitions, conversion linking
3.7	Replace SF behavior inside /api/preregister and /api/mobcontact	local persistence + compatibility response contract
3.8	Build CRM communications layer	confirmation/reply/escalation/lead acknowledgement templates + delivery tracking
3.9	Build Customer Service UX flows in Filament	triage-first inbox, saved filters/views, one-click assignment, keyboard-first actions
3.10	Build Operations UX flows in Filament	certificate review workspace, side-by-side doc preview, bulk approve/reject with safeguards
3.11	Build user-friendly data presentation	clear status badges, SLA countdown chips, sticky context panels, empty/error states
3.12	Add accessibility and usability standards	WCAG AA contrast, visible focus states, responsive layout, low-click-path interaction
3.13	Build CRM analytics widgets	queue depth, aging buckets, first-response SLA, resolution SLA, reopened ticket rate
3.14	UAT with CS/Ops and iterate UX	task-based usability tests, record friction points, apply prioritized fixes

#	Task	Notes
3.15	Finalize CRM playbooks and training assets	SOPs, macro templates, escalation ladder, handoff guidelines

Phase 4 - Platform Admin and Health (Week 1-2)

#	Task	Notes
4.1	Build <code>PlatformSettingsPage</code>	runtime toggles/config
4.2	Build <code>PolicyResource</code> + versioning workflow	draft/publish/rollback
4.3	Build <code>SystemHealthPage</code>	DB/queue/mail/storage/FCM/Stripe checks
4.4	Build <code>QueueOpsPage</code>	retry/discard/requeue
4.5	Build dashboard widgets for SLA/health	
4.6	Add immutable audit logs for admin actions	

Phase 5 - Salesforce Cutover (Week 2)

#	Task	Notes
5.1	Execute contract parity suite in staging	approval, lead, support, user lookup paths
5.2	Run shadow-read/dual-observe period (no SF write dependence)	compare outcomes and latency without SF as source of truth
5.3	Cut traffic to internal modules only	freeze Salesforce integration endpoints from operational use
5.4	Keep <code>/api/mobcontact</code> as compatibility alias	same response contract, mapped to local ticket creation
5.5	Remove SF runtime dependencies from code	middleware, routes, controllers, listeners, helper calls
5.6	Remove SF persistence artifacts	deprecate then drop <code>sales_kitchens</code> after verification window
5.7	Remove SF secrets and rotate exposed credentials	env + secret store cleanup, key rotation evidence

#	Task	Notes
5.8	Verify zero Salesforce traffic in production logs	monitor for 7 days, alert on any outbound SF call
5.9	Sign-off on no-impact outcomes	no data loss, no workflow regression, SLA and response times within target

Phase 5.5 - Deployment and Release Tasks (parallel to cutover)

#	Task	Notes
5.5.1	Build container image(s) for backend and ops	same artifact, separate services recommended
5.5.2	Deploy ops-admin to staging under same-domain path	e.g. staging.mitabl.com/admin/* , restricted access
5.5.3	Configure ingress and WAF rules	tighter rules for admin than public/api
5.5.4	Configure horizontal scaling policies	api and admin tuned separately
5.5.5	Queue worker deployment separation	dedicate workers for CRM notifications/escalations
5.5.6	Add health probes	liveness/readiness/startup probes per service
5.5.7	Add zero-downtime DB migration steps	pre-deploy, deploy, post-deploy phases
5.5.8	Configure centralized logs/metrics/traces	tagged by marketing , api , admin
5.5.9	Define rollback runbook	route rollback + migration rollback strategy
5.5.10	Production cutover rehearsal	full dry-run in staging with timing and owners

Phase 6 - Hardening (Week 2)

#	Task	Notes
6.1	Redis queue + Horizon	async stability

#	Task	Notes
6.2	Add intake throttles and abuse controls	API + captcha if needed
6.3	Feature tests: certificate flow	approve/reject/resubmit
6.4	Feature tests: ticket lifecycle + SLA	
6.5	Feature tests: policy permissions	role boundary checks
6.6	Load tests on ticket intake and admin lists	
6.7	Disaster recovery runbook + backup validation	

Strict Dependency Timeline (Predecessors + Critical Path)

Execution rule:

- A task cannot start until all listed predecessors are completed.
- Any task marked `Critical Path = Yes` blocks production cutover if incomplete.

Milestone Gates

Gate	Definition	Blocks
G0	Phase 0 foundations complete	All build phases
G0.5	Platform hardening gate complete	Phase 1+ feature delivery
G1	Admin foundation complete	Core resources/CRM UI
G3	CRM functionally complete + UX accepted	Salesforce cutover
G5	Salesforce eradication verification complete	Production sign-off
G6	Hardening regression/perf gates complete	Project closure

Dependency Matrix

ID	Task	Predecessors	Critical Path
0.1-0.7	Repository/deployment foundations	none	Yes
0.5.1	PHP 8.2+ baseline	0.1-0.7	Yes
0.5.2	Laravel 8 -> 11	0.5.1	Yes
0.5.3	Dart 3+ upgrade	0.1-0.7	No

ID	Task	Predecessors	Critical Path
0.5.4	Enforce <code>QUEUE_CONNECTION=redis</code>	0.5.2	Yes
0.5.5	Deploy queue workers/Horizon	0.5.4	Yes
0.5.6	Switch cache backend to Redis	0.5.2	Yes
0.5.7	Add discovery caching	0.5.6	Yes
0.5.8	Remove double-fetch pagination patterns	0.5.2	Yes
0.5.9	Cache invalidation hooks	0.5.7	Yes
0.5.10	Extract <code>StripeTrait -> PaymentService</code>	0.5.2	Yes
0.5.11	Service layer extraction	0.5.2	Yes
0.5.12	Remove hardcoded SF secrets from source	0.5.2	Yes
0.5.13	Async/cache observability	0.5.4, 0.5.6	Yes
0.5.14	Hardening regression tests	0.5.7, 0.5.9, 0.5.10, 0.5.11	Yes
0.5.15	Hardening exit criteria gate	0.5.4-0.5.14	Yes
1.1	Install Filament	0.5.15	Yes
1.2	Scaffold panel	1.1	Yes
1.3	<code>AdminUser</code> model/migration/seeder	1.2	Yes
1.4	Spatie permission + role matrix	1.3	Yes
1.5	Panel branding	1.2	No
1.6-1.10	Core migrations (cert/user/ticket/lead/settings)	1.3	Yes
2.1	<code>CertificateResource</code>	1.4, 1.6	Yes
2.2	Certificate notifications	2.1, 0.5.5	Yes
2.3	<code>UserResource</code>	1.4, 1.7	Yes
2.4	<code>MikitchnResource</code>	1.4, 2.1	Yes
2.5	<code>OrderResource</code> + refund controls	1.4, 0.5.10	Yes
2.6	<code>PromoCodeResource</code>	1.4	No

ID	Task	Predecessors	Critical Path
3.1	Ticket schema/index readiness	1.8	Yes
3.2	SupportTicketResource workflow	3.1, 1.4	Yes
3.3	Support APIs (create/get/reply)	3.1, 0.5.11	Yes
3.4	Spam/abuse protection	3.3	Yes
3.5	SLA timers/escalation jobs	3.1, 0.5.5	Yes
3.6	PreRegistrationResource workflow	1.9, 1.4	Yes
3.7	Replace SF behavior in prereg/mobcontact	3.3, 3.6	Yes
3.8	CRM communications layer	3.2, 3.3, 0.5.5	Yes
3.9	CS UX flows	3.2	Yes
3.10	Ops UX flows	2.1, 2.4, 3.2	Yes
3.11	User-friendly UI states/components	3.9, 3.10	Yes
3.12	Accessibility/usability standards	3.11	Yes
3.13	CRM analytics widgets	3.1, 3.5	No
3.14	UAT for CS/Ops workflows	3.8, 3.11, 3.12	Yes
3.15	SOPs/training/playbooks	3.14	Yes
4.1-4.6	Platform admin/health modules	1.10, 1.4, 0.5.13	No
5.1	Staging contract parity suite	2.1-2.5, 3.1-3.15	Yes
5.2	Shadow-read/dual-observe period	5.1	Yes
5.3	Cut traffic to internal modules only	5.2	Yes
5.4	/api/mobcontact compatibility alias live	3.7, 5.3	Yes
5.5	Remove SF runtime dependencies from code	5.3	Yes
5.6	Remove SF persistence artifacts	5.5	Yes
5.7	Remove SF secrets + credential rotation	5.5	Yes
5.8	Zero SF traffic verification (7 days)	5.5, 5.7	Yes
5.9	No-impact final sign-off	5.8, 3.14	Yes

ID	Task	Predecessors	Critical Path
6.1-6.7	Final hardening validation	5.9	Yes

Critical Path Sequence

- 0.1-0.7 -> 0.5.1 -> 0.5.2 -> 0.5.4 -> 0.5.5 -> 0.5.6 -> 0.5.7 -> 0.5.9 -> 0.5.10 -> 0.5.11 -> 0.5.14 -> 0.5.15
- 1.1 -> 1.2 -> 1.3 -> 1.4 -> 1.6-1.10
- 2.1 -> 2.2 and 2.3 -> 2.4 -> 2.5
- 3.1 -> 3.2 and 3.3 -> 3.4 and 3.5 -> 3.7 -> 3.8 -> 3.9 and 3.10 -> 3.11 -> 3.12 -> 3.14 -> 3.15
- 5.1 -> 5.2 -> 5.3 -> 5.5 -> 5.6 and 5.7 -> 5.8 -> 5.9
- 6.1-6.7

Parallel Workstreams (Allowed)

Workstream	Can run in parallel after
Mobile Dart 3 migration (0.5.3)	0.1-0.7
Panel branding (1.5)	1.2
PromoCode resource (2.6)	1.4
Platform admin health pages (4.1-4.6)	1.10 , 1.4 , 0.5.13
CRM analytics widgets (3.13)	3.1 , 3.5

CRM Functional Specification (Detailed)

CRM Agent UX Requirements (Customer Service + Operations)

- Single-screen triage layout:
 - left: queue/inbox with fast filters,
 - center: conversation/ticket timeline,
 - right: contextual profile (user, kitchen, orders, payment summary).
- Productivity-first interactions:
 - keyboard shortcuts for assign/status/reply,
 - saved views (My Queue , Unassigned , SLA Risk , Ops Certificates Pending),
 - bulk actions with confirmation for high-risk changes.
- Clear visual hierarchy:
 - color-safe status badges,
 - SLA countdown chips (on track , at risk , breached),

- unread/new indicators and sticky action bar.
- Friction reduction:
 - canned macros/templates with variables,
 - auto-suggest related tickets and similar past resolutions,
 - one-click jump links to user/order/kitchen resources.
- Safe operations UX:
 - irreversible actions require typed confirmation,
 - optimistic UI only where idempotent; otherwise explicit completion feedback,
 - collision detection with "updated by another agent" conflict resolution modal.
- Accessibility and device support:
 - WCAG AA contrast baseline,
 - full keyboard navigability and visible focus,
 - usable 1280px desktop baseline with responsive fallback for tablet.

Ticket Lifecycle

- open -> in_progress -> pending_user -> resolved -> closed
- spam terminal state for abuse/noise
- Reopen rules:
 - allowed within configurable window after resolve
 - requires reason

Assignment and Routing

- Auto-routing by category/priority/policy.
- Manual reassignment with audit reason.
- Escalation ladder (time-based and severity-based).

SLA

- Define per category/priority:
 - first response target
 - resolution target
- Automated breach flags, notifications, and escalation.

Communication

- Outbound email confirmation and reply notifications.
- Internal notes never exposed to requester.
- Optional later extension: in-app ticket thread for authenticated users.

Knowledge and Macros

- Canned responses/templates by category.
- Tag-based recommendations for frequent issue patterns.

Data Governance

- PII masking in list views where not required.
- Ticket/message retention policy.
- Export with role-based redaction.

Platform Admin Functional Specification (Detailed)

Configurations

- Runtime settings registry:
 - support defaults
 - onboarding policy toggles
 - maintenance/operational flags
- Change workflow:
 - validate
 - approve (if high-risk)
 - activate with audit trail

Policies

- Versioned JSON policy documents.
- Effective dating and rollback.
- Diff view before publishing.

Health Checks

- Liveness/readiness endpoints.
- Scheduled synthetic checks for key flows:
 - ticket intake
 - email dispatch
 - queue processing
 - storage read/write

Operations

- Failed job triage with bulk retry.
- Integration failure dashboard.
- Incident mode flag (degraded operation handling).

Security Admin

- Admin session policy (expiry, re-auth for sensitive actions).
- API key/secret rotation runbook.
- Access review report and dormant admin detection.

Dependency Summary

```
# required
composer require filament/filament:"^3.0"
composer require spatie/laravel-permission

# recommended
composer require laravel/horizon
composer require owen-it/laravel-auditing
```

No separate hosting tier is required for the internal portal.

Security Considerations

Risk	Mitigation
Admin session and mobile JWT cross-contamination	Separate <code>AdminUser</code> guard/provider
Brute-force on <code>/admin/login</code>	Login throttling + optional IP allowlisting + MFA
Unauthorized approval/refund actions	Permission-gated actions + policy checks + step-up confirmation
Sensitive docs exposure	Signed temporary URLs + strict storage access policy
Ticket spam abuse	Rate limits + honeypot/captcha on public forms + spam status
Duplicate action execution	Idempotency keys + DB transaction locks
Lost auditability	Immutable audit log with actor, before/after, correlation ID
PII overexposure	Field-level masking and export redaction by role

Testing, UAT, and Cutover Acceptance

Must-pass automated tests

- Certificate approve/reject/resubmit paths
- Ticket intake, assignment, SLA transitions, close/reopen
- Role boundary tests for all high-risk resources/actions
- Policy publish/rollback behavior
- Health check and queue operational pages permissions

UAT scenarios

- Customer service day-in-the-life workflows
- Operations certificate triage under load
- Platform admin emergency runbook execution

Cutover acceptance criteria

- 100% parity for legacy SF flows used in production
- No data loss for prereg/support submissions
- Ticket SLA timers accurate in production timezone
- Zero unresolved P1 security findings